

Java SOLID Principles - Cheat Sheet

SOLID is five principles for writing OOP code that doesn't fall apart when requirements change. When a class feels painful to modify or test, one of these is usually being violated.

S - Single Responsibility Principle

A class should have one reason to change.

Bad:

```
public class Invoice {  
    public double calculateTotal() { ... }  
    public void saveToDatabase() { ... }  
    public void sendEmail() { ... }  
}
```

Three reasons this class can change: tax rules change, DB schema changes, email format changes. Split it.

Good:

```
public class Invoice {  
    public double calculateTotal() {  
        // ...  
    }  
}  
  
public class InvoiceRepository {  
    public void save(Invoice invoice) {  
        // ...  
    }  
}  
  
public class InvoiceMailer {  
    public void send(Invoice invoice) {  
        // ...  
    }  
}
```

Rule of thumb: if you keep saying “this class does X **and** Y”, split it.

O - Open/Closed Principle

Open for extension, closed for modification. Add new behavior without touching old code.

Bad:

```
public class DiscountService {
    public double getDiscount(String customerType) {
        if (customerType.equals("REGULAR")) {
            return 0.05;
        }
        if (customerType.equals("PREMIUM")) {
            return 0.10;
        }
        if (customerType.equals("VIP")) {
            return 0.20;
        }
        return 0;
    }
}
```

Every new customer type forces you to edit this method, with the risk of breaking the existing branches.

Good:

```
public interface Customer {
    double discount();
}

public class Regular implements Customer {
    public double discount() {
        return 0.05;
    }
}

public class Premium implements Customer {
    public double discount() {
        return 0.10;
    }
}

public class Vip implements Customer {
    public double discount() {
        return 0.20;
    }
}
```

New customer type? Add a new class. The old ones don't move.

L - Liskov Substitution Principle

A subclass must work anywhere the parent works, without surprising the caller.

The classic broken example:

```
public class Rectangle {
    protected int width;
    protected int height;

    public void setWidth(int w) {
        this.width = w;
    }

    public void setHeight(int h) {
        this.height = h;
    }

    public int area() {
        return width * height;
    }
}

public class Square extends Rectangle {
    @Override
    public void setWidth(int w) {
        this.width = w;
        this.height = w;
    }

    @Override
    public void setHeight(int h) {
        this.width = h;
        this.height = h;
    }
}
```

Looks innocent until a caller does `setWidth(5)` then `setHeight(10)` on a `Rectangle` reference and expects area 50. With a `Square`, the caller gets 100. The subclass quietly broke an assumption the parent made.

Smells that point to an LSP violation: - Overridden methods that throw

`UnsupportedOperationException` - Subclass adds preconditions the parent didn't have - Subclass returns weaker guarantees than the parent - Callers using `instanceof` to special-case a subclass

I - Interface Segregation Principle

Don't force a class to implement methods it doesn't need.

Bad:

```

public interface Worker {
    void work();
    void eat();
    void sleep();
}

public class Robot implements Worker {
    public void work() {
        // ...
    }

    public void eat() {
        throw new UnsupportedOperationException();
    }

    public void sleep() {
        throw new UnsupportedOperationException();
    }
}

```

Now `Robot` is lying about what it can do. Anyone calling `eat()` on a `Worker` reference gets a runtime surprise.

Good:

```

public interface Workable {
    void work();
}

public interface Eatable {
    void eat();
}

public interface Sleepable {
    void sleep();
}

public class Human implements Workable, Eatable, Sleepable {
    public void work() { ... }
    public void eat() { ... }
    public void sleep() { ... }
}

public class Robot implements Workable {
    public void work() { ... }
}

```

Small focused interfaces beat one fat one. Java lets a class implement as many as it needs.

D - Dependency Inversion Principle

High-level code should depend on abstractions, not on concrete classes.

Bad:

```
public class OrderService {
    private MySQLOrderRepository repo = new MySQLOrderRepository();

    public void place(Order o) {
        repo.save(o);
    }
}
```

`OrderService` is married to MySQL. Switching to Postgres means editing the service. Unit testing means a real DB connection.

Good:

```
public interface OrderRepository {
    void save(Order o);
}

public class OrderService {
    private final OrderRepository repo;

    public OrderService(OrderRepository repo) {
        this.repo = repo;
    }

    public void place(Order o) {
        repo.save(o);
    }
}
```

Now you can pass in a MySQL repo, a Postgres repo, or a mock for tests. The service doesn't care.

This is the principle that makes DI frameworks like Spring and Guice actually useful.

Quick Reference

Letter	Name	One-line rule
S	Single Responsibility	One class, one reason to change
O	Open/Closed	Extend without editing existing code
L	Liskov Substitution	Subclass must behave like its parent
I	Interface Segregation	Split fat interfaces into small ones
D	Dependency Inversion	Depend on interfaces, not concrete classes

How They Connect

SRP and ISP are the same idea at different scales. SRP for classes, ISP for interfaces. Both say: keep things focused.

OCP and DIP travel together. You extend without modifying (OCP) because your code depends on abstractions (DIP). Drop DIP and OCP becomes painful to pull off.

LSP is the safety net under inheritance. It catches the case where the code compiles but the behavior quietly breaks.

Common Gotchas

- SRP is about reasons to change. A class with twenty cohesive methods can still follow it if they all serve one responsibility.
- OCP applies when adding features. Bug fixes can still edit existing code freely.
- LSP violations often hide behind `instanceof` checks or special-cased subclasses in caller code. Treat those as a smell.
- ISP starts mattering once an interface grows past 4-5 methods. Small interfaces stay clean on their own.
- DIP works without a DI framework. A constructor that takes an interface parameter is already following it.